

4. Header Linked List

–Insert At END.

Program:

```
void insertAtEnd(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr) {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }

    newNode->data = data;
    newNode->next = nullptr;

    // Start traversal from head
    Node *temp = head;

    // Move to the last node
    while (temp->next != nullptr)
    {
        temp = temp->next;
    }

    // Link the new node
    temp->next = newNode;

    // Increment internal count
    head->data++;

    cout << data << " inserted at the end." << endl;
}

...case 2:
    cout << "Enter data to insert: ";
    cin >> data;
    insertAtEnd(data);
    break;
```

Program Analysis

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of *newNode* will be equal to the size of a *Node* structure and *newNode* is declared as a pointer to *Node* structure.

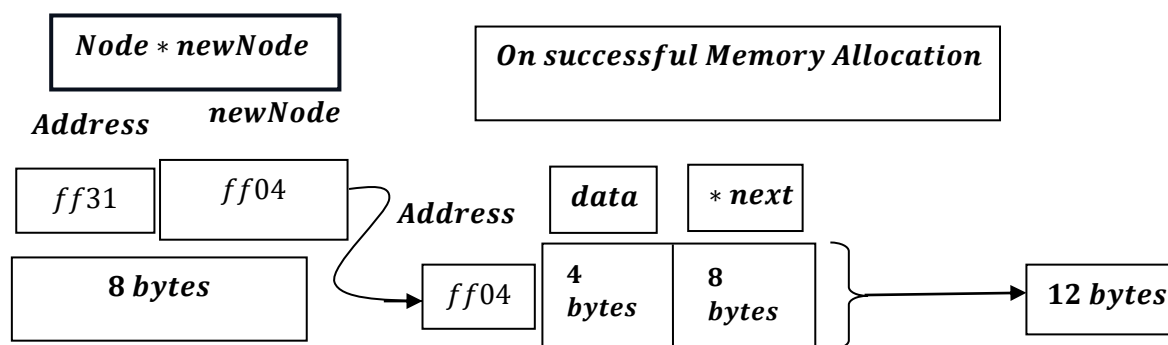
→ The size of a pointer (*newNode*) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the *Node* structure itself is determined by the sum of the sizes of its members:

→ *data*(an integer) is typically 4 bytes.

→ *next*(a pointer to another *Node*) is typically 8 bytes.

→ Therefore ,the size of the *Node* structure is typically 12 bytes(4 bytes for *data* + 8 bytes for *next*) .



```

if (newNode == nullptr)
{
    cout << "Error: Memory allocation failed." << endl;
    return;
}

```

Now newNode = ff04, hence → Memory Allocation is Successful !

if newNode = NULL ,then :

Message → "Error: Memory Allocation for Header Failed."

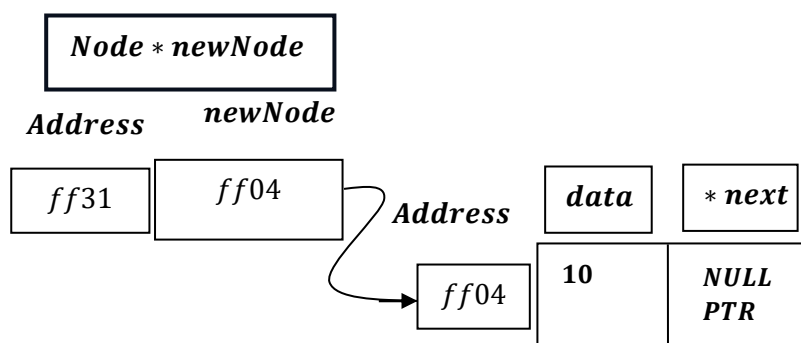
And then reurn will return void and exit from the module (function).

```

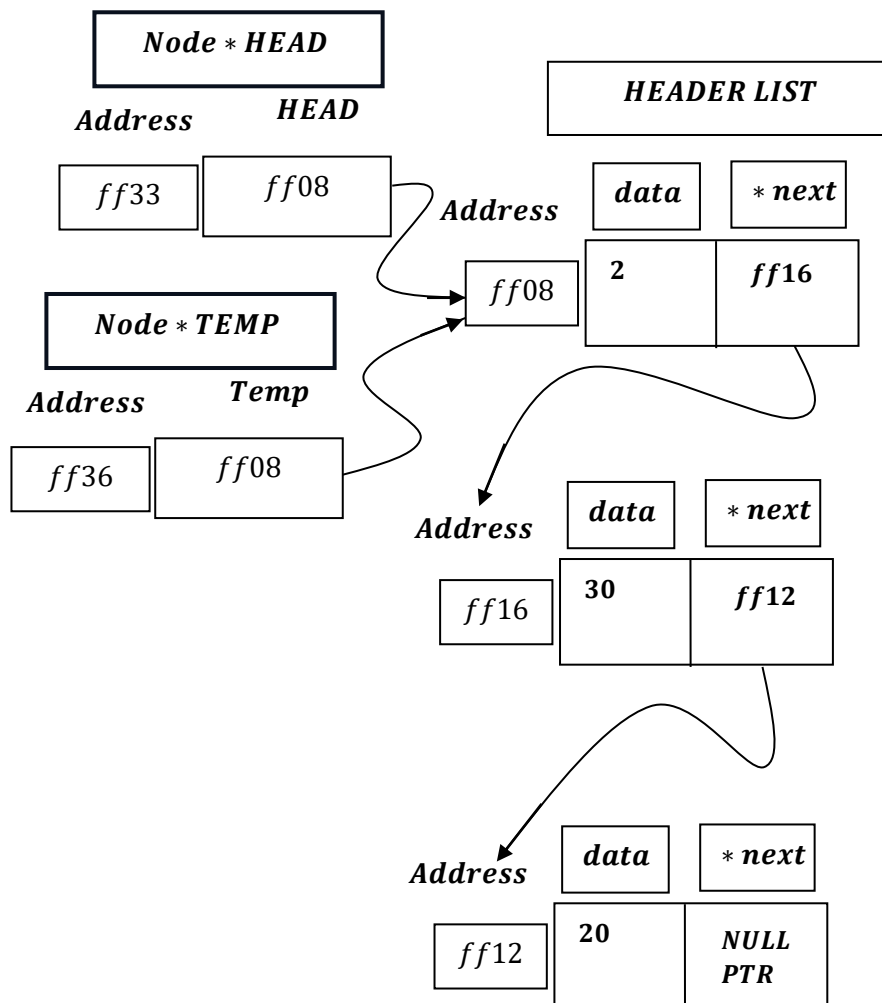
newNode->data = data;
newNode->next = nullptr;

```

Let data = 10, next = nullptr, then:



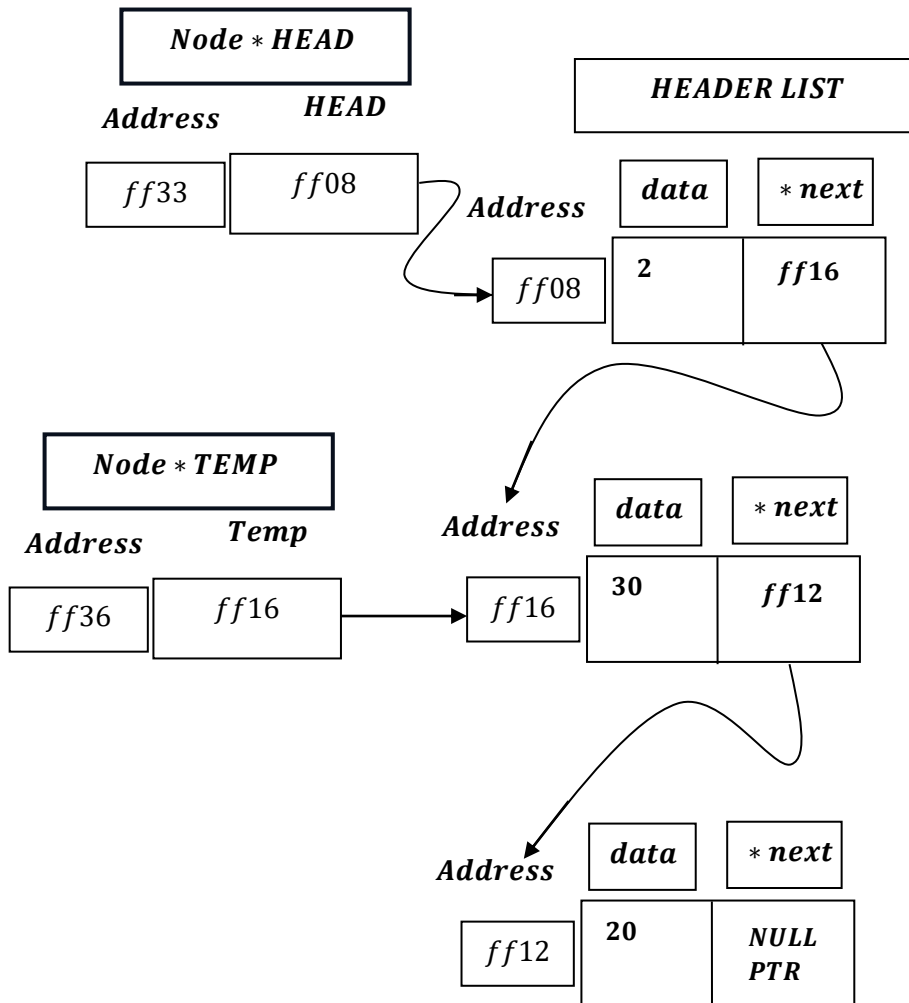
```
Node *temp = head;
```



```
while (temp->next != nullptr)
{
    temp = temp->next;
}
```

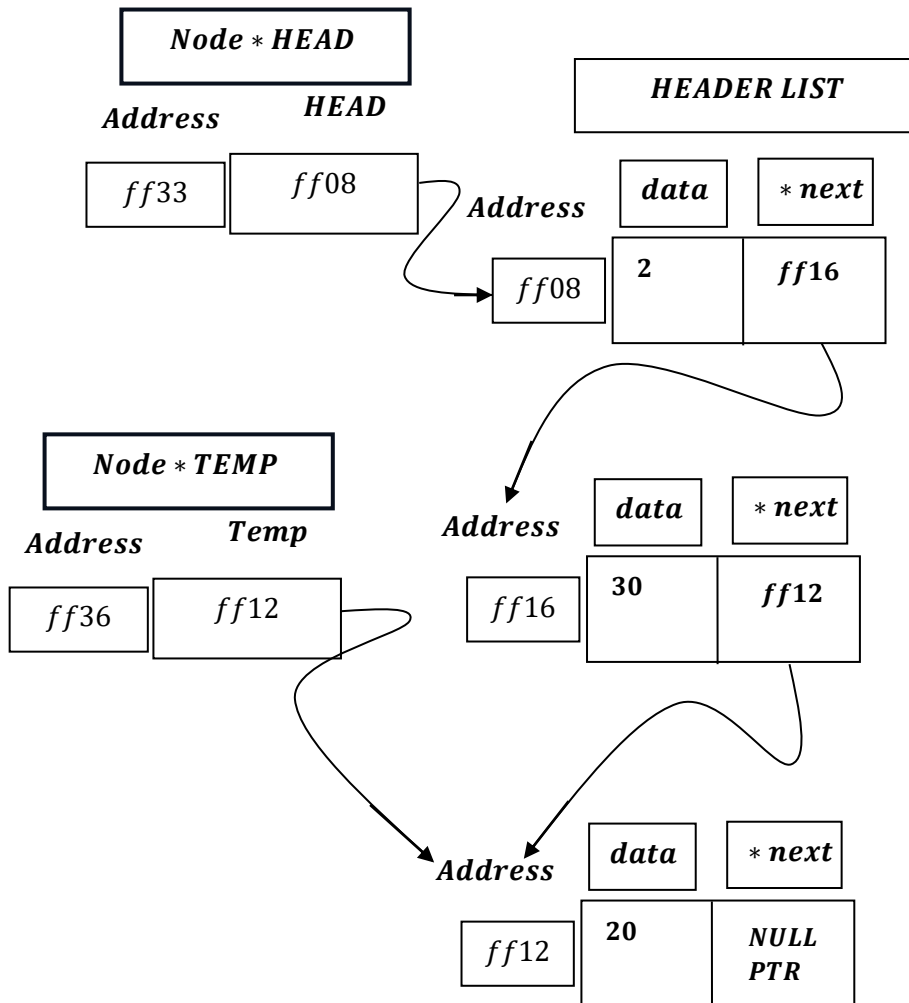
$temp \rightarrow next = ff16 \neq NULLPTR :$

$TEMP = TEMP \rightarrow next = ff16$



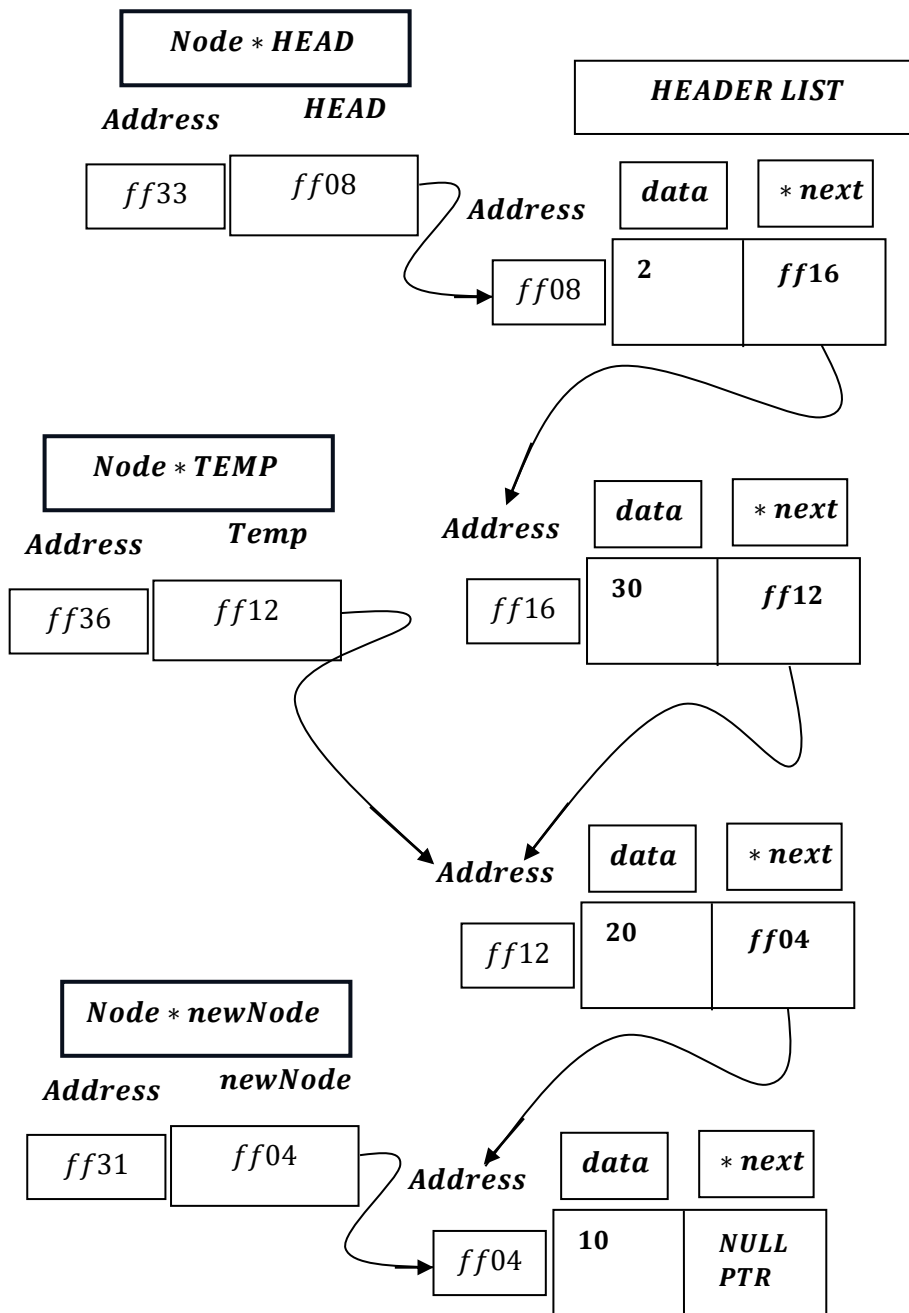
$temp \rightarrow next = ff12 \neq NULLPTR :$

$TEMP = TEMP \rightarrow next = ff12$

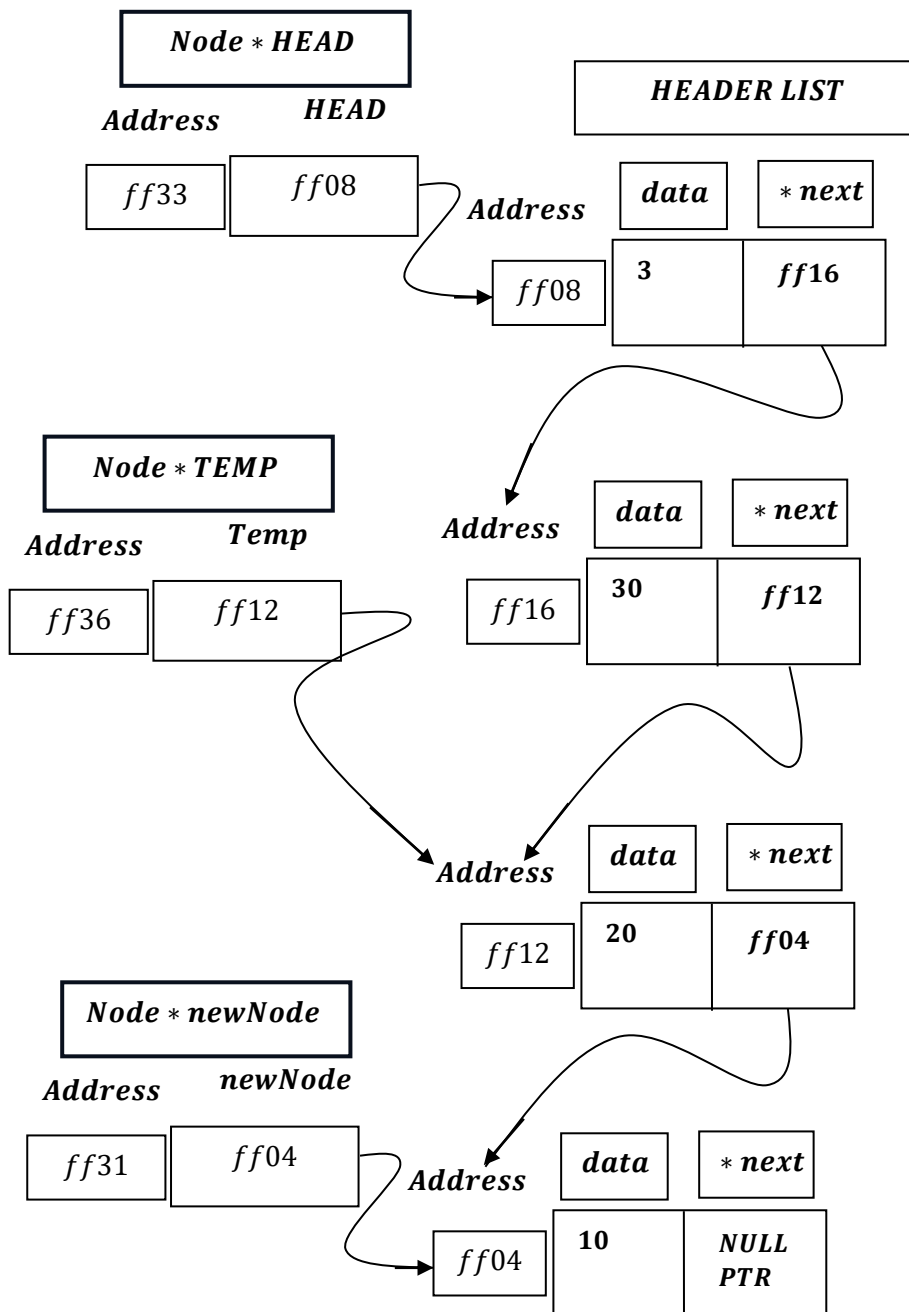


$temp \rightarrow next = NULLPTR \neq NULLPTR$ is false, hence exit from loop.

```
temp->next = newNode;
```



```
head->data++;
```



And, we get our final list:

